

NewToon 온보딩 → 초기 뉴스 추천 API 연동 마스터 프롬프트

현재 제작되어 있는 **NewToon 모바일 프론트엔드 UI/UX와 Prototype**을 유지하면서, 온보딩에서 사용자가 선택한 관심 카테고리 3개를 실제 ALGORITHM_AGENT 백엔드에 전송하고, 백엔드가 반환한 뉴스 4개를 Home Lobby의 “오늘의 NewToon” 1개 + “관심 분야 뉴스” 3개로 표시하도록 프론트엔드 로직을 구현해줘.

이번 작업의 핵심은 UI를 새롭게 재설계하는 것이 아니라,

사용자 선택 → API Request → Loading → API Response → Home Rendering

까지의 실제 데이터 연결 흐름을 완성하는 것이다.

중요:

- 기존에 제작된 NewToon 디자인 시스템을 유지한다.
- 기존 알 캐릭터 디자인과 Home Layout을 유지한다.
- Backend는 이미 구현된 ALGORITHM_AGENT를 사용한다.
- Mock 뉴스, 하드코딩된 추천 뉴스, 임의의 가짜 API Response를 사용하지 않는다.
- 실제 Backend API 계약을 절대 임의로 변경하지 않는다.

1. 전체 사용자 흐름

다음 흐름을 구현한다.

```
Splash
↓
뉴스 수준 선택
↓
관심 뉴스 카테고리 선택
↓
카테고리 정확히 3개 선택
↓
“내 NewToon 시작하기”
↓
POST /api/v1/recommendations/initial
↓
추천 뉴스 Loading
↓
Backend Response
↓
response.data.items 확인
↓
items[0] → 오늘의 NewToon
```

```
items[1] → 관심 분야 뉴스 1
items[2] → 관심 분야 뉴스 2
items[3] → 관심 분야 뉴스 3
↓
Home Lobby Rendering
```

온보딩 완료 이후 Home에 표시되는 뉴스는 반드시 Backend Response를 사용한다.

임의로 뉴스 제목이나 추천 결과를 생성하지 않는다.

2. 공식 관심 카테고리

온보딩 관심 분야 화면에는 정확히 다음 10개의 카테고리를 표시한다.

정치
경제
사회
국제
과학, 기술
환경, 기후
문화, 연예
스포츠
건강, 생활
교육, 청소년

사용자 화면에 표시되는 문자열과 Backend API로 전달되는 문자열을 **완전히 동일하게 사용한다.**

예:

과학, 기술
환경, 기후
문화, 연예
건강, 생활
교육, 청소년

다음과 같이 임의로 변경하지 않는다.

과학·기술	X
과학기술	X
환경·기후	X
환경기후	X

문화·연예	X
문화연예	X
건강·생활	X
건강생활	X
교육·청소년	X
교육청소년	X

침표와 띄어쓰기를 포함한 Backend 공식 문자열을 그대로 유지한다.

3. 카테고리 선택 UI

사용자는 관심 분야를 정확히 3개 선택해야 한다.

기존 카테고리 선택 화면의 디자인을 유지하면서 아래 인터랙션을 구현한다.

초기 상태:

0 / 3 선택

첫 번째 선택:

1 / 3 선택

두 번째 선택:

2 / 3 선택

세 번째 선택:

3 / 3 선택 완료 ✓

선택된 Category Card:

- NewToon Blue Border
- Soft Blue Background
- Check Icon
- 제목 Bold
- 선택 여부가 명확하게 보이게 한다.

선택되지 않은 Category Card:

- White Background
- Light Gray 또는 Beige Border
- Dark Navy Text

카테고리 선택값은 프론트 내부의 `selectedCategories` 상태에 저장한다.

예:

```
selectedCategories = [  
  "경제",  
  "과학, 기술",  
  "국제"  
];
```

카테고리를 클릭할 때마다 Backend에 요청하지 않는다.

온보딩이 완료되는 마지막 CTA를 눌렀을 때 한 번만 Initial Recommendation API를 호출한다.

4. 카테고리 3개 선택 제한

카테고리는 정확히 3개까지만 선택할 수 있다.

이미 3개를 선택한 상태에서 네 번째 카테고리를 클릭하면 새로운 카테고리를 추가하지 않는다.

Toast:

관심 분야는 3개까지 선택할 수 있어요.

Secondary Text:

다른 분야를 선택하려면 선택한 항목 하나를 먼저 해제해주세요.

기존 선택값은 그대로 유지한다.

5. CTA 활성화

카테고리가 3개 미만이면:

내 NewToon 시작하기

버튼은 Disabled 상태다.

Disabled:

- Muted Background
- Shadow 없음
- 클릭 불가

정확히 3개를 선택하면 버튼을 활성화한다.

Active:

- NewToon Blue Background
- White Text
- Bold 700
- Height 56px
- Radius 16px

버튼:

내 NewToon 시작하기

6. user_id와 session_id

Initial Recommendation API에는 다음 정보가 필요하다.

```
user_id
session_id
interests
limit
```

`user_id`에는 실제 현재 로그인 사용자 식별값을 사용한다.

`session_id`에는 현재 추천 세션의 고유값을 사용한다.

프론트에서 이미 사용자 및 세션 관리 기능이 존재한다면 해당 값을 그대로 사용한다.

아직 사용자 인증 또는 Session 관리 방식이 완성되지 않은 경우:

- Backend 계약을 임의로 변경하지 않는다.
 - API 연동 코드에서 실제 userId / sessionId가 주입될 위치를 명확하게 분리한다.
 - 임의의 고정값을 실제 서비스 코드에 영구적으로 하드코딩하지 않는다.
-

7. 초기 추천 API

사용자가 카테고리 3개를 선택한 뒤:

내 NewToon 시작하기

버튼을 누르면 다음 API를 호출한다.

```
POST /api/v1/recommendations/initial
```

Header:

```
Content-Type: application/json
```

Request Body:

```
{
  "user_id": "현재 로그인 사용자의 ID",
  "session_id": "현재 추천 세션 ID",
  "interests": [
    "경제",
    "과학, 기술",
    "국제"
  ],
  "limit": 4
}
```

실제 코드 구조는 다음 형태로 구현한다.

```
const requestBody = {
  user_id: userId,
  session_id: sessionId,
  interests: selectedCategories,
  limit: 4,
};

const response = await api.post(
  "/api/v1/recommendations/initial",
  requestBody
);
```

중요:

`interests`에는 사용자가 선택한 공식 카테고리 문자열 3개를 그대로 전달한다.

현재 API 계약에 존재하지 않는 다음 필드를 임의로 보내지 않는다.

```
news_level
category
news_id
difficulty
age
userLevel
```

특히 온보딩에서 사용자의 뉴스 이해 수준을 별도로 선택하더라도 현재 `/initial` API에는 `news_level` 을 전달하지 않는다.

해당 수준 데이터는 프론트 내부 상태 또는 향후 API 확장용 데이터로만 유지한다.

8. API 요청 Loading State

`내 NewToon 시작하기` 버튼 클릭 후 바로 Home으로 이동하지 않는다.

먼저 Initial Recommendation Loading State를 보여준다.

버튼:

맞춤 뉴스를 준비하고 있어요...

작은 Spinner를 함께 표시한다.

Loading 중:

- 버튼 중복 클릭 금지
- Category 변경 금지
- 중복 API 요청 금지
- 기존 사용자 선택값 유지

알 캐릭터는 다음과 같은 행동을 한다.

- 돋보기로 뉴스를 찾는 모습
- 여러 뉴스 카드를 살펴보는 모습
- 사용자의 관심 분야를 정리하는 모습

Loading Text:

관심 분야에 맞는 오늘의 뉴스를 찾고 있어요.

Optional Secondary:

여러 뉴스 중 지금 읽기 좋은 뉴스를 고르고 있어요.

9. Initial Response 구조

API Response는 배열 자체가 아니다.

추천 뉴스 배열은 반드시:

```
response.data.items
```

에서 읽는다.

전체 구조:

```
interface InitialRecommendationResponse {  
  request_id: string;  
  items: RecommendationItem[];  
  count: number;  
  generated_at: string;  
}
```

따라서:

```
const items = response.data.items;
```

로 접근한다.

다음을 사용하지 않는다.

```
response.data[0]    // 금지  
response.items      // 금지  
response.news       // 금지  
response.recommendations // 금지
```

정확한 접근은:

```
response.data.items
```

이다.

10. Recommendation Item 구조

각 Item은 다음 구조를 가진다.


```
interface RecommendationItem {
  recommendation_type: RecommendationType;
  layout_role: LayoutRole;
  similarity_score: number | null;
  recommendation_reason: string;
  source: string | null;
  news: NewsData;
}
```

News:

```
interface NewsData {
  title: string;
  content: string;
  news_url: string;
  thum_url: string | null;
  broadcast_date: string;
  cluster: string;
}
```

11. 필드 접근 경로

각 뉴스의 필드는 반드시 다음 위치에서 읽는다.

제목:

```
item.news.title
```

본문:

```
item.news.content
```

출처:

```
item.source
```

날짜:

```
item.news.broadcast_date
```

썸네일:

item.news.thum_url

원문 URL:

item.news.news_url

중복 뉴스 그룹:

item.news.cluster

추천 이유:

item.recommendation_reason

Role:

item.layout_role

유사도:

item.similarity_score

12. 중요: source 위치

`source` 는 `news` 객체 안에 존재하지 않는다.

올바른 접근:

item.source

잘못된 접근:

item.news.source

또한 제목과 본문 역시 Item Root에 존재하지 않는다.

잘못된 접근:

```
item.title  
item.content
```

올바른 접근:

```
item.news.title  
item.news.content
```

13. 추천 순서

Backend는 Initial Recommendation의 `items` 배열 순서를 반드시 다음과 같이 반환한다.

```
items[0] = MAIN  
items[1] = RELATED  
items[2] = RELATED  
items[3] = EXPLORE
```

RELATED 2개는 MAIN 뉴스와의 similarity 내림차순이다.

EXPLORE는 추천 순위 4위 뉴스가 아니라 다른 주제를 발견하기 위해 선택된 탐색용 뉴스다.

Frontend에서 배열을 다시 정렬하지 않는다.

다음과 같이 사용한다.

```
const items = response.data.items;  
  
const mainNews = items[0];  
const interestNews = items.slice(1, 4);
```

동시에 `layout_role`도 검사한다.

```
mainNews.layout_role === "MAIN"  
  
interestNews[0].layout_role === "RELATED"  
interestNews[1].layout_role === "RELATED"  
interestNews[2].layout_role === "EXPLORE"
```

14. Home Lobby Rendering

API가 성공하면 기존 Home Lobby UI에 실제 데이터를 렌더링한다.

Home은 크게 두 영역으로 구성한다.

오늘의 NewToon
+
관심 분야 뉴스

15. 오늘의 NewToon Hero Card

`response.data.items[0]` 을 Home 상단의 가장 큰 Hero Card에 사용한다.

즉:

```
const mainNews = response.data.items[0];
```

Label

오늘의 NewToon

Thumbnail

`mainNews.news.thum_url`

Title

`mainNews.news.title`

Source

`mainNews.source`

Date

`mainNews.news.broadcast_date`

Recommendation Reason

mainNews.recommendation_reason

추천 이유는 너무 길게 표시하지 않고 카드 아래 짧은 보조 설명으로 사용한다.

예:

관심 분야와 가장 관련성이 높은 오늘의 뉴스예요.

CTA

6컷으로 보기

기존 Hero Card 디자인의 크기와 중요도를 유지한다.

16. 날짜 표시

Backend가 주는 날짜:

2026-08-13T12:30:00+09:00

을 그대로 길게 표시하지 않는다.

사용자 화면에서는 읽기 쉽게 변환한다.

예:

2026.08.13

또는:

8월 13일

가능하면 한국 사용자에게 자연스러운 형식을 사용한다.

데이터 원본값 자체를 수정해서 저장하지 않고 화면 표시 단계에서만 Formatting한다.

17. 관심 분야 뉴스 3개

Home Hero Card 아래 섹션 제목:

관심 분야 뉴스

보조 문구:

내가 선택한 관심사를 바탕으로 골라봤어.

다음 3개 Item을 순서대로 표시한다.

```
items[1]
items[2]
items[3]
```

재정렬하지 않는다.

18. 관심 뉴스 1

```
const news1 = items[1];
```

Role:

RELATED

표시:

- Thumbnail
- Title
- Source
- Date
- Recommendation Reason

필드:

```
news1.news.thum_url
news1.news.title
news1.source
news1.news.broadcast_date
news1.recommendation_reason
```

추천 이유 예:

메인 뉴스와 함께 보면 이해하기 좋아요.

19. 관심 뉴스 2

```
const news2 = items[2];
```

Role:

RELATED

동일한 Card Component를 사용한다.

첫 번째 RELATED보다 similarity가 낮지만 Backend에서 순서가 이미 정해져 있으므로 그대로 보여준다.

Frontend에서 similarity를 기준으로 다시 정렬하지 않는다.

20. 관심 뉴스 3

```
const news3 = items[3];
```

Role:

EXPLORE

EXPLORE는 RELATED와 의미가 다르므로 사용자에게 살짝 다른 Label을 사용할 수 있다.

예:

새로운 주제

또는:

이런 뉴스도 있어요

설명:

평소 관심 분야 밖의 새로운 주제를 발견해보세요.

하지만 기존 **관심 분야 뉴스** 섹션 안의 세 번째 카드 위치를 유지한다.

Backend 배열 위치를 변경하지 않는다.

21. Home Card에서 표시하지 말아야 할 가짜 정보

현재 API에는 다음 데이터가 존재하지 않는다.

```
category
news_id
공식자료 개수
언론사 개수
뉴스 난이도
예상 읽기 시간
```

따라서 Backend에서 받지 않은 값을 API 데이터인 것처럼 만들어서 표시하지 않는다.

예를 들어 다음 내용을 임의로 생성하지 않는다.

```
경제
공식자료 2개
언론사 3곳
뉴스 입문
약 3분
```

기존 UI에 해당 영역이 있다면 실제 API에서 확보 가능한 다음 정보로 대체한다.

```
source
broadcast_date
recommendation_reason
layout_role
```

예:

Reuters · 8월 13일

관심 분야와 가장 관련성이 높은 오늘의 뉴스

22. Thumbnail null 처리

`thum_url` 은 null일 수 있다.

따라서:


```
const imageUrl =  
  item.news.thum_url ?? DEFAULT_NEWS_IMAGE;
```

형태로 처리한다.

`thum_url === null` 이면 프로젝트 내부에 이미 존재하는 **NewToon 기본 뉴스 Placeholder Image**를 사용한다.

Placeholder는:

- 단순한 뉴스 아이콘
- NewToon Blue / Cream 디자인
- 알 캐릭터를 과도하게 크게 넣지 않음

으로 구성한다.

이미지가 없다는 이유로 Card Layout이 깨지지 않게 한다.

23. source null 처리

`source` 도 null일 수 있다.

다음 중 하나의 방식으로 처리한다.

권장:

```
const sourceLabel =  
  item.source ?? "출처 미제공";
```

화면:

출처 미제공

또는 공간이 자연스럽지 않으면 Source 영역 자체를 숨긴다.

`null`, `undefined`, 빈 문자열을 사용자에게 그대로 표시하지 않는다.

24. Response 유효성 검사

API 성공 후 다음 조건을 확인한다.

```
response.data.items
```

가 존재하는가?

그리고:

```
response.data.items.length
```

가 4개인가?

또한:

```
items[0] MAIN  
items[1] RELATED  
items[2] RELATED  
items[3] EXPLORE
```

인지 확인한다.

정상이라면 Home을 Render한다.

응답 구조가 예상과 다르면 Mock News로 대체하지 않는다.

Error State를 표시한다.

25. Initial API 실패 UX

API 요청 실패 시 Home으로 이동하지 않는다.

현재 온보딩 화면 또는 추천 Loading 화면을 유지한다.

Error Card 또는 Toast:

맞춤 뉴스를 불러오지 못했어요.

Secondary:

인터넷 연결을 확인하거나 잠시 후 다시 시도해주세요.

CTA:

다시 시도

다시 시도 클릭:

POST /api/v1/recommendations/initial

재호출.

중요:

- 사용자가 선택한 카테고리 3개 유지
- user_id 유지
- session_id 유지
- 처음부터 다시 선택하게 하지 않는다.

26. Empty / Invalid Response

HTTP 요청은 성공했지만:

items가 없음
items가 0개
items가 4개 미만
layout_role이 예상과 다름

등의 문제가 있으면 다음 UI를 표시한다.

추천 뉴스를 준비하지 못했어요.

Secondary:

잠시 후 다시 시도해주세요.

CTA:

추천 다시 받기

가짜 뉴스를 대신 렌더링하지 않는다.

27. “6컷으로 보기” 데이터 전달

현재 Backend Response에는 `news_id`가 존재하지 않는다.

따라서 Frontend에서 `news_id`를 임의 생성하지 않는다.

개별 뉴스 식별 및 다음 화면 전달에는:

```
item.news.news_url
```

을 사용한다.

Hero News에서 `6컷으로 보기` 클릭 시:

```
const selectedItem = mainNews;

navigate("/webtoon", {
  state: {
    newsUrl: selectedItem.news.news_url,
    news: selectedItem.news,
    source: selectedItem.source,
    recommendationReason:
      selectedItem.recommendation_reason
  }
});
```

형태로 다음 단계로 전달할 수 있다.

정확한 Router 구조는 기존 프로젝트 구조를 따른다.

28. cluster 사용 주의

다음 값:

```
item.news.cluster
```

는 개별 기사 ID가 아니다.

동일 또는 유사 뉴스의 중복 그룹을 나타내는 Cluster ID다.

따라서:

```
news_id
article_id
unique news identifier
```

처럼 사용하지 않는다.

기사 전달과 식별에는 현재:

news_url

을 사용한다.

29. Recommendation Reason 활용

Backend의:

item.recommendation_reason

을 사용자가 추천 이유를 이해하는 데 활용한다.

MAIN 예:

왜 이 뉴스일까?

관심 분야와 가장 관련성이 높은 오늘의 뉴스예요.

RELATED 예:

메인 뉴스와 관련성이 높은 기사예요.

EXPLORE 예:

새로운 주제를 발견할 수 있도록 골라봤어요.

텍스트가 길 경우 1~2줄까지만 표시하고 필요하면 말줄임표를 사용한다.

30. similarity_score 사용

RELATED Item에서는:

item.similarity_score

가 숫자로 올 수 있다.

하지만 MVP Home에서 **69.9% 유사** 같은 기술 수치를 반드시 사용자에게 보여줄 필요는 없다.

사용자 경험에서는 추천 이유를 우선 표시한다.

개발 및 Debug 용도로는 값을 유지한다.

다음 값은 null일 수도 있다.

```
MAIN → null
EXPLORE → null
```

따라서 null-safe하게 처리한다.

31. Home Loading Skeleton

Initial API가 호출된 후 Home Card가 나타나기 전까지 Skeleton 상태를 만들 수 있다.

Hero Skeleton:

- Thumbnail 영역
- 제목 2줄
- Source / Date
- CTA 자리

Interest News Skeleton:

- 작은 썸네일
- 제목 2줄
- Source / Date

알 캐릭터 옆:

“딱 맞는 뉴스를 찾고 있어!”

단, Loading이 매우 짧아도 UI가 깨지지 않게 한다.

32. Home Layout 최종 구성

Home은 다음 구조로 정리한다.

[Header]

NewToon

연속 학습

XP / Profile

↓

[Greeting]

오늘은 어떤 세상을 알아볼까?



[오늘의 NewToon]

items[0] = MAIN

큰 Hero Card

- 실제 Thumbnail
- 실제 Title
- 실제 Source
- 실제 Date
- Recommendation Reason
- 6컷으로 보기



[관심 분야 뉴스]

items[1] RELATED

items[2] RELATED

items[3] EXPLORE

각 뉴스 카드:

- Thumbnail
- Title
- Source
- Date
- 필요시 Recommendation Reason



[Bottom Navigation]

홈

탐색

레벨

MY

33. 기존 Home의 UI Style 유지

현재 NewToon Home에서 사용하고 있는:

- NewToon Blue
- Cream / Yellow Accent
- Rounded Card
- Bold Title
- 알 캐릭터
- 밝고 친근한 10대 뉴스 앱 분위기
- 하단 홈 / 탐색 / 레벨 / MY

구조를 그대로 유지한다.

이번 수정에서 UI를 완전히 새로 디자인하지 않는다.

핵심은 **정적인 가짜 뉴스 카드를 실제 Backend Data 기반 컴포넌트로 변경하는 것이다.**

34. TypeScript 타입

다음 타입을 Backend 계약 그대로 사용한다.

```
type Category =  
  | "정치"  
  | "경제"  
  | "사회"  
  | "국제"  
  | "과학, 기술"  
  | "환경, 기후"  
  | "문화, 연예"  
  | "스포츠"  
  | "건강, 생활"  
  | "교육, 청소년";  
  
type LayoutRole =  
  | "MAIN"  
  | "RELATED"  
  | "EXPLORE";  
  
type RecommendationType =  
  | "INITIAL"  
  | "SIMILAR"  
  | "SERENDIPITY";  
  
interface InitialRecommendationRequest {  
  user_id: string;
```



```

session_id: string;
interests: [Category, Category, Category];
limit: 4;
}

interface NewsData {
  title: string;
  content: string;
  news_url: string;
  thum_url: string | null;
  broadcast_date: string;
  cluster: string;
}

interface RecommendationItem {
  recommendation_type: RecommendationType;
  layout_role: LayoutRole;
  similarity_score: number | null;
  recommendation_reason: string;
  source: string | null;
  news: NewsData;
}

interface InitialRecommendationResponse {
  request_id: string;
  items: RecommendationItem[];
  count: number;
  generated_at: string;
}

```

Backend Field Name을 임의로 camelCase로 변경하지 않는다.

예:

```

news_url
broadcast_date
thum_url
recommendation_reason
layout_role

```

를 그대로 사용한다.

35. API 관련 절대 금지사항

다음 사항을 반드시 지킨다.

1. Mock News를 사용하지 않는다.
2. Backend 응답 실패 시 임의 뉴스로 대체하지 않는다.
3. API Field Name을 임의 변경하지 않는다.
4. `items` 배열을 Frontend에서 다시 정렬하지 않는다.
5. MAIN / RELATED / RELATED / EXPLORE 순서를 변경하지 않는다.
6. `category`를 임의 생성하지 않는다.
7. `news_id`를 임의 생성하지 않는다.
8. `cluster`를 `news_id`처럼 사용하지 않는다.
9. 기사 식별에는 `news_url`을 사용한다.
10. `source`를 `item.news.source`로 접근하지 않는다.
11. `source`는 반드시 `item.source`를 사용한다.
12. "과학·기술"을 Backend에 보내지 않는다.
13. "과학, 기술"을 정확히 사용한다.
14. "환경·기후" 대신 "환경, 기후"를 사용한다.
15. "문화·연예" 대신 "문화, 연예"를 사용한다.
16. "건강·생활" 대신 "건강, 생활"을 사용한다.
17. "교육·청소년" 대신 "교육, 청소년"을 사용한다.
18. 현재 API 계약에 없는 `news_level`을 Request Body에 추가하지 않는다.
19. Loading 중 API를 중복 호출하지 않는다.
20. 실제 Backend Data가 오기 전에 Home 뉴스 내용을 임의 생성하지 않는다.

36. 개발용 전체 데이터 흐름

다음 Data Flow를 기준으로 구현한다.

[ONBOARDING]

사용자 뉴스 수준 선택

※ 현재 API에는 전송하지 않음

↓

관심 카테고리 선택

경제

과학, 기술

국제

↓

Frontend State

```
selectedCategories = [  
  "경제",  
  "과학, 기술",  
  "국제"  
]
```

↓

3 / 3 선택 완료

↓

“내 NewToon 시작하기”

↓

Request Body 생성

```
{  
  user_id,  
  session_id,  
  interests: selectedCategories,  
  limit: 4  
}
```

↓

POST
/api/v1/recommendations/initial

↓

LOADING

↓

Backend Response

response.data.items

↓

items[0]

MAIN

↓

오늘의 NewToon Hero

```
items[1]
RELATED
```

```
items[2]
RELATED
```

```
items[3]
EXPLORE
```

↓

관심 분야 뉴스 3개

↓

HOME Rendering

37. Prototype에서 표현할 상태

Figma 또는 프론트 Prototype에서 최소 다음 상태를 만든다.

```
Interest_0_Selected
↓
Interest_1_Selected
↓
Interest_2_Selected
↓
Interest_3_Selected
↓
Initial_Request>Loading
↓
Initial_Request_Success
↓
Home_With_API_Data
```

오류:

```
Initial_Request_Error
↓
Retry
```

Invalid Response:

Initial_Response_Invalid
↓
Retry

Thumbnail null:

News_Card_Default_Image

Source null:

News_Card_No_Source

38. 발표용 핵심 Demo Flow

해커톤 발표에서는 다음 흐름이 자연스럽게 보여야 한다.

관심 뉴스 설정

경제
과학, 기술
국제

↓

3 / 3 선택 완료

↓

내 NewToon 시작하기

↓

“맞춤 뉴스를 준비하고 있어요...”

↓

실제 Backend 추천 결과 수신

↓

HOME

[오늘의 NewToon]

MAIN 뉴스

↓

[관심 분야 뉴스]

RELATED

RELATED

EXPLORE

↓

사용자가 MAIN 선택

↓

6컷으로 보기

↓

해당 MAIN 뉴스의
news_url을 다음 화면으로 전달

39. 최종 성공 기준

이번 작업이 성공적으로 구현되었다고 판단하려면 다음 조건을 모두 만족해야 한다.

- 사용자가 공식 카테고리 중 정확히 3개를 선택할 수 있다.
- 카테고리 문자열이 Backend 공식 문자열과 완전히 동일하다.
- 3개 선택 후 `내 NewToon 시작하기` 가 활성화된다.
- 버튼 클릭 시 `POST /api/v1/recommendations/initial` 을 호출한다.
- `user_id, session_id, interests, limit`을 정확히 전달한다.
- Loading 중 중복 요청을 막는다.
- 실제 추천 배열은 `response.data.items` 에서 읽는다.
- `items[0]` 은 Home의 `오늘의 NewToon` 이 된다.
- `items[1]~items[3]` 은 `관심 분야 뉴스` 가 된다.
- MAIN / RELATED / RELATED / EXPLORE 순서를 유지한다.
- 뉴스 제목은 `item.news.title` 을 사용한다.
- 출처는 `item.source` 를 사용한다.
- 날짜는 `item.news.broadcast_date` 를 사용한다.
- 썸네일은 `item.news.thum_url` 을 사용한다.
- 원문 주소는 `item.news.news_url` 을 사용한다.

- source와 thum_url의 null 상태를 처리한다.
- Backend에서 제공하지 않는 category와 news_id를 생성하지 않는다.
- 추천 결과를 임의로 재정렬하지 않는다.
- API Error 시 Mock News를 보여주지 않는다.
- Error 및 Retry UI를 보여준다.
- 6컷으로 보기 클릭 시 해당 뉴스의 news_url 을 다음 단계로 전달한다.
- 현재 NewToon의 디자인과 알 캐릭터 Identity를 유지한다.

이번 수정의 최종 목표는 다음 한 문장이다.

사용자가 선택한 관심 카테고리 3개가 실제 추천 API에 전달되고, ALGORITHM_AGENT가 반환한 실제 추천 결과 4개가 순서 그대로 NewToon Home의 오늘의 뉴스와 관심 뉴스로 연결되는 완전한 Frontend-Backend 데이터 흐름을 구현한다.